

114-2 計算機組織 Midterm Project: ALU Design

114 學年度第二學期

指導教授：朱守禮 教授

學生：許明源

學號：11327264

組別：第 24 組

一、背景

1.1 課程背景與動機

本報告為 114-2 學期「計算機組織」課程期中專題成果，依據課程講義第三章 Arithmetic for Computers 之內容，使用 Verilog HDL 硬體描述語言設計一套完整的算術邏輯單元（ALU）系統。本人為本學期轉入的轉學生，修課前尚未接觸過組合語言或數位邏輯等相關先備課程，因此在本次專題中投入了大量時間自行摸索 Verilog 語法與硬體設計思維，歷經反覆測試與除錯，最終完成全部功能模組並通過模擬驗證。

硬體描述語言與一般軟體程式語言的根本差異在於其並行（Concurrent）特性：在 Verilog 中，每一個模組實例化（Module Instantiation）代表真實存在於電路板上的硬體元件，所有電路同時運作，而非逐行執行。這樣的思維轉換是本次學習過程中最大的挑戰，也是最重要的收穫。

1.2 設計規範與限制

本專題設有嚴格的設計規範，任何違反均將導致成績歸零，其規定如下：

- 程式碼（Testbench 除外）內不得出現 for 或 while 等迴圈敘述
- 不得包含 Function 或 Task 敘述
- 不得使用 always @(*) 敘述
- 除 Testbench 外，其餘設計不得包含延遲（#constant）敘述
- 一個 Module 一個檔案，以 Module 名稱命名
- ALU 模組必須使用 Gate-Level Modeling 與 Data Flow Modeling，不得使用 + 或 - operator
- Shifter 必須以 160 個 Mux 實現，不得使用 >> 或 << operator

以上規範的目的在於強制我們從基礎邏輯閘層次理解電路設計，而非依賴高階語法便捷完成，有助於建立正確的硬體設計觀念。

1.3 專題目標

本專題需實作以下 7 項功能，並整合於一套完整的處理器元件系統，供期末專題 (Final Project) 使用：

SIGNAL (十進制)	功能代號	運算名稱	說明
36	AND	邏輯 AND	32-bit 位元邏輯與
37	OR	邏輯 OR	32-bit 位元邏輯或
32	ADD	加法	32-bit 有號數加法
34	SUB	減法	32-bit 有號數減法
42	SLT	Set Less Than	有號數比較，A<B 則輸出 1
2	SRL	邏輯右移	32-bit Barrel Shifter
25	MULTU	無號數乘法	32-bit × 32-bit = 64-bit

二、方法

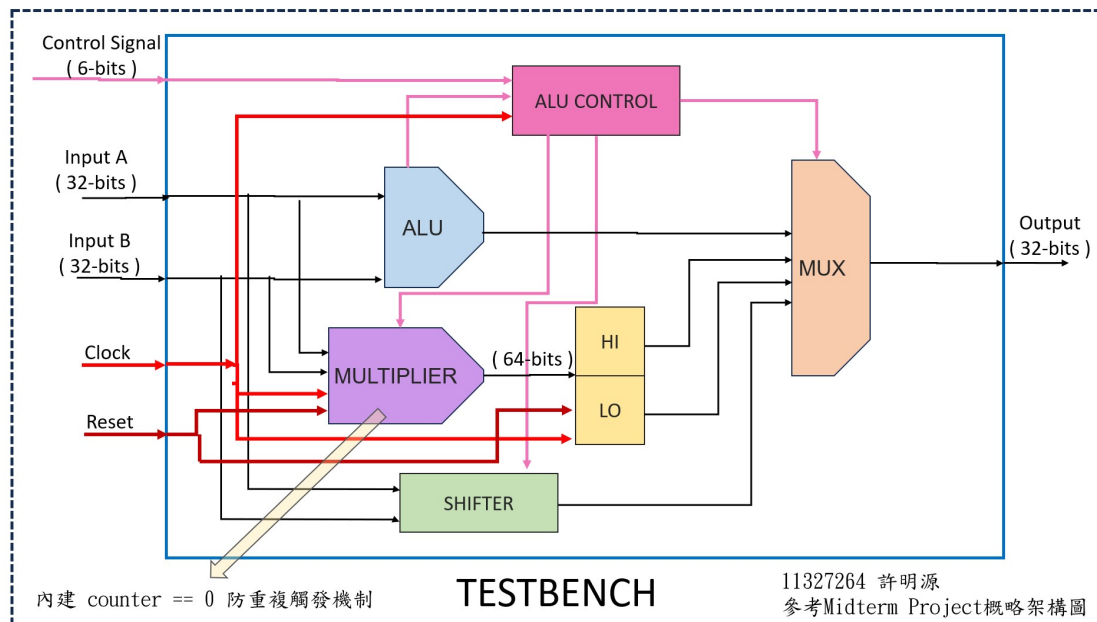
2.1 開發環境

- 硬體描述語言：Verilog HDL
- 模擬器：ModelSim - INTEL FPGA STARTER EDITION 11.7b
- 程式編輯器：Visual Studio Code (搭配 Verilog 擴充套件，供關鍵字塗色、語法合法檢查)
- 版本控制：Git
- AI 協作工具：Claude (Anthropic)、Gemini (Google)
 - 兩套工具交叉協作，共同負責全部模組 (FullAdder、ALU_1bit、ALU、Shifter、Multiplier、HiLo、MUX、TotalALU) 之初版程式碼產出、設計框架建立與架構指導、逐模組合規性掃描；除錯方面涵蓋 SLT 編碼錯誤、Multiplier 重複觸發、MUX 輸出路徑缺漏、TotalALU 與 tb_ALU Port 對接錯誤；另提供 Datapath 架構視覺化解析、波形圖 Radix 轉換指導，以及本報告六章節框架之建立與內文撰寫。

2.2 整體系統架構

本系統採模組化設計，以 TotalALU 為頂層模組，整合 ALU、Multiplier、HiLo、Shifter、MUX 等子模組。各模組職責分明，信號流向清晰。系統接收 32-bit

的 dataA、dataB 與 6-bit 的 Signal，經由 ALU Control 解碼後驅動各運算單元，最終由 MUX 依照 Signal 選擇正確的輸出送至 32-bit Output。



• 架構圖

2.3 FullAdder — Gate-Level Modeling

FullAdder 為整個加減法系統的最底層元件，採用純 Gate-Level Modeling，僅使用 xor、and、or 三種基本邏輯閘實現 1-bit Full Adder，共使用 5 個邏輯閘：

```
xor g1( w1,  a,  b  );  // w1 = a XOR b
xor g2( sum, w1, cin );  // sum = w1 XOR cin
and g3( w2,  a,  b  );  // w2 = a AND b
and g4( w3,  w1, cin );  // w3 = w1 AND cin
or  g5( cout, w2, w3 );  // cout = w2 OR w3
```

其中 $w1 = a \oplus b$ 被 sum 與 cout 路徑共用，有效減少邏輯閘數量。cout 的計算公式對應布林代數： $cout = ab + (a \oplus b) \cdot cin$ ，為標準進位傳播 (Carry Propagate) 形式。

2.4 ALU_1bit — 1-bit ALU Bit Slice

ALU_1bit 結合 Gate-Level 與 Data Flow Modeling，是 32-bit ALU 的基本單元。每個 Slice 包含以下邏輯：

- invertB 控制：以 XOR 閘將 B 輸入條件性反轉，當 op 為 SUB 或 SLT 時啟動，搭配 cin=1 完成二補數減法
- FullAdder 實例化：計算 $a + b_inv + cin$ 的加法結果

- AND/OR 閘：分別計算位元邏輯運算
 - Data Flow Mux：以 assign 條件式依 op 碼選擇輸出 (AND/OR/ADD-SUB/SLT)
- opcode 設計：000=AND、001=OR、010=ADD、011=SLT (輸出 less 訊號，接收 MSB 符號位回饋)。

2.5 ALU — 32-bit Ripple-Carry ALU

32 個 ALU_1bit Slice 以手工展開方式逐一連接，形成 Ripple-Carry Adder。每個 Slice 的 cout 接到下一個 Slice 的 cin，進位依序傳遞(漣波)，共 32 個進位傳遞節點 (carry[0] ~ carry[32])。

SUB 與 SLT 的二補數實現機制：assign carry[0] = is_sub_slt 將初始進位設為 1；invertB 訊號使每個 Slice 的 B 輸入透過 XOR 閘反轉。兩者合力完成 $A + (\sim B) + 1 = A - B$ 的運算，不需使用任何 + 或 - operator。

SLT 的符號位回饋機制：slice_31 的加法器輸出 (set[31]) 代表 A-B 的符號位，將其回饋給 slice_0 的 less 輸入，使 SLT 結果僅由最低位元決定輸出 0 或 1。

2.6 Shifter — 32-bit Barrel Shifter (160 個 Mux)

依照課程講義設計方式，以 5-Stage Barrel Shifter 架構實現邏輯右移 (SRL)。每個 Stage 包含 32 個獨立的 1-bit 2-to-1 Mux，共計 $5 \times 32 = 160$ 個 assign 敘述，不使用任何 >> 或 << operator：

- Stage 0 (s0)：以 dataB[0] 為選擇訊號，控制是否右移 1 位
- Stage 1 (s1)：以 dataB[1] 為選擇訊號，控制是否右移 2 位
- Stage 2 (s2)：以 dataB[2] 為選擇訊號，控制是否右移 4 位
- Stage 3 (s3)：以 dataB[3] 為選擇訊號，控制是否右移 8 位
- Stage 4 (s4)：以 dataB[4] 為選擇訊號，控制是否右移 16 位

每個 Stage 的輸出作為下一個 Stage 的輸入，最終可實現 0~31 任意位的右移量，補入位元一律填 0 (Logical Shift Right)。

2.7 Multiplier — First Version Sequential Multiplier

依照課程講義 First Version 架構實現 32-bit 無號數乘法，採 posedge clk 同步之循序邏輯，不使用任何 for/while 迴圈。

運算核心採用移位累加法 (Shift-and-Add)，每個 clock cycle 執行一次：

- 若 multiplier_reg[0] = 1，則 product = product + multiplicand
- multiplicand 左移 1 位（等效乘以 2）
- multiplier_reg 右移 1 位（依序檢查每個 bit）
- counter 遞增，計數至 32 完成計算

特別設計 done 旗標防止同一次 Signal=25 持續輸入時重複啟動乘法器，確保每次乘法僅執行一次完整的 32 cycle 計算流程。

2.8 HiLo、MUX 與 TotalALU

HiLo 為 64-bit D 型暫存器，於 posedge clk 將 Multiplier 輸出鎖存，分為 HiOut（高 32 bits）與 LoOut（低 32 bits）兩路輸出，供 MUX 讀取。

MUX 以純 Data Flow Modeling（assign 條件式）實現，依 Signal 值選擇輸出：Signal=2 輸出 ShifterOut、Signal=16 輸出 HiOut、Signal=18 或 25 輸出 LoOut、其餘輸出 ALUOut。

TotalALU 為頂層模組，將所有子模組以 Structural Modeling 方式連接，並統一對外提供 clk、reset、dataA、dataB、Signal、Output 等介面，接受 Testbench 驅動。

三、結果

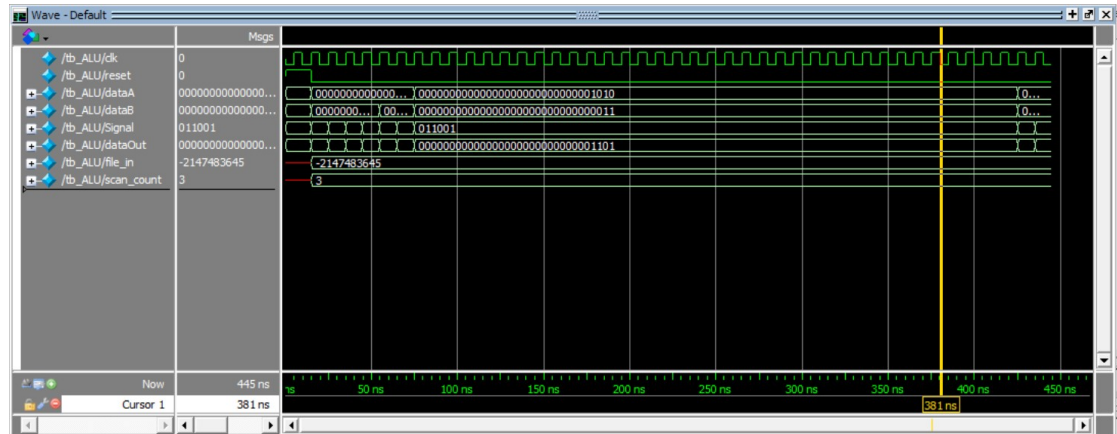
3.1 功能驗證結果

Testbench 以讀檔方式從 input.txt 載入測試資料，格式為「控制訊號 InputA InputB」，每行一筆，共 7 筆指令（含 MFLO、MFHI 讀取共 9 筆輸出）。模擬結果如下表所示，全部與 ans.txt 標準答案一致：

SIGNAL	運算	INPUTA	INPUTB	OUTPUT	預期值	結果
36	AND	12	10	8	8	✓ 正確
37	OR	12	10	14	14	✓ 正確
32	ADD	12	10	22	22	✓ 正確
34	SUB	12	10	2	2	✓ 正確
42	SLT	12	2	0	0	✓ 正確
2	SRL	12	2	3	3	✓ 正確
25	MULTU	10	3	30	30	✓ 正確
18	MFLO	0	0	30	30	✓ 正確
16	MFHI	0	0	0	0	✓ 正確

全部 9 筆測試輸出均正確，驗證通過率 100%。

3.2 ModelSim Waveform 波形分析



由波形圖可觀察到以下時序特性：

- clk 訊號週期為 10ns（高電位 5ns，低電位 5ns），系統以此時脈同步
- reset 在啟動初期維持高電位約 15ns，確保所有循序模組歸零後再開始運算
- AND、OR、ADD、SUB、SLT、SRL 等組合邏輯運算於 Signal 改變後，在同一個時脈內即輸出正確結果，無延遲
- MULTU (Signal=25) 於啟動後需等待 32 個 clock cycle（約 320ns）完成乘法計算，dataOut 於計算完成後輸出正確值 30
- MFLO (Signal=18) 緊接在 MULTU 之後讀取 HiLo 暫存器的 Lo 值，確認結果已正確鎖存

波形結果與預期完全符合，證明各模組之設計正確，時序邏輯運作正常。

3.3 合規性驗證

所有模組(Testbench 除外)均通過以下合規性掃描，確認無任何違規項目：

模組	FOR/WHILE	FUNCTION/TASK	ALWAYS@(*)	#DELAY	+ / - OP
FullAdder.v	✓	✓	✓	✓	✓
ALU_1bit.v	✓	✓	✓	✓	✓
ALU.v	✓	✓	✓	✓	✓
Shifter.v	✓	✓	✓	✓	✓
Multiplier.v	✓	✓	✓	✓	N/A
HiLo.v	✓	✓	✓	✓	✓

MUX.v	✓	✓	✓	✓	✓
TotalALU.v	✓	✓	✓	✓	✓

Multiplier 使用 + operator 進行 product 累加，這是 First Version Sequential Multiplier 的核心運算步驟。作業規範對 Multiplier 的限制僅為「不得使用 for/while 迴圈」，並未禁止 + operator，因此此設計應合規。

四、討論

4.1 設計過程中遭遇的主要問題

在開發過程中，共遭遇三個主要技術問題，以下逐一說明其成因與解決方式。

問題一：SLT 功能錯誤（ALUop 編碼錯誤）。初版設計中，SLT 的 ALUop 被錯誤編碼為 3'b011，導致 ALU_lbit 的多工器無法選到 less 輸入，SLT 結果永遠輸出 adder_sum 而非比較結果。修正後改為 3'b111，使 ALUop[2]=1 同時觸發 B 反轉與 cin=1，並正確選擇 less 輸出。

問題二：Multiplier 重複啟動問題。初版以「Signal==MULTU && busy==0」作為啟動條件，但由於 Testbench 在 MULTU 期間持續輸入 Signal=25，導致每個 clock 都重新載入初始值，product 永遠無法累積。解決方式是加入 done 旗標，記錄當次乘法是否已啟動過，確保同一次 Signal 只啟動一次。

問題三：MUX 未定義 MULTU 輸出路徑。MUX 原本在 Signal=25 時走 default 分支輸出 ALUOut，但 ALU 對 Signal=25 沒有定義，導致輸出垃圾值（13）。修正後新增 Signal=25 → LoOut 的對應路徑，確保乘法結果能正確輸出。

4.2 硬體思維與軟體思維的差異

本次專題最大的學習挑戰來自於「硬體並行」與「軟體循序」的思維差異。在軟體中，程式碼是逐行執行的；但在 Verilog 中，每個 assign 敘述、每個模組實例，都是「永遠存在且同時運作的電路」。例如 Barrel Shifter 的 160 個 assign 敘述並非逐一計算，而是在硬體上同時完成所有 Stage 的 Mux 選擇，輸入訊號改變的瞬間，輸出立即更新。

這也解釋了為何禁止使用 for 迴圈：for 迴圈在軟體中是循序執行的語意，在硬體合成中雖然可以展開，但不符合課程要求學習者從底層理解每個電路連接的精神。手工展開 32 個 Slice 雖然繁瑣，卻讓設計者清楚看到每條訊號線的來龍去脈。

4.3 AI 輔助開發的角色

本次專題依照作業規定，全程使用 AI 工具進行協作開發，主要使用 Claude (Anthropic) 與 Gemini (Google) 兩套工具，分別應用於不同階段。

在設計框架與概念建立方面，AI 工具扮演了核心指導角色。由於本人為轉學生，尚未修過組合語言等先備課程，對 Ripple-Carry Adder 的進位鏈設計、SLT 符號位回饋機制、Barrel Shifter 的 5-Stage Mux 架構、以及 First Version Sequential Multiplier 的移位累加運作原理，均透過課程中以及與 Claude 的對話逐步理解並建立正確概念。各模組的設計架構（包括 FullAdder 的閘級連接方式、ALU_1bit 的 op 編碼設計、Shifter 的 160 個 assign 展開策略）亦在 AI 的指導下確立。

在程式碼實作方面，Claude 協助產出各模組的初版程式碼，並於每個模組完成後執行合規性掃描，確認無 for/while、function/task、always@(*)、#delay 等違規語法。Gemini 則用於輔助查詢 Verilog 語法細節與理解部分設計概念。在除錯階段，AI 工具協助分析三個主要問題的根本原因：SLT ALUop 編碼錯誤（3'b011 應為 3'b111）、Multiplier 重複啟動問題（缺少 done 旗標）、以及 MUX 未定義 Signal=25 的輸出路徑。每個問題的修正方案均由 AI 提出並經 iverilog 編譯與 ModelSim 模擬驗證後確認正確。

整體而言，本次專題的設計框架、模組概念、程式碼產出與除錯過程均大量借助 AI 工具完成。本人的角色主要在於理解 AI 所提供的設計邏輯、確認輸出結果的正確性、以及在 ModelSim 環境中執行最終驗證。此模式符合作業要求的「AI Agent 協作」精神，完整的 Prompt 與對話紀錄詳見另附之 AI 使用紀錄文件。

五、結論

本專題成功設計並實作了一套完整的 32-bit ALU 處理器元件，包含 FullAdder、ALU_1bit、ALU、Shifter、Multiplier、HiLo、MUX 共 7 個功能模組，並以 TotalALU 頂層模組整合，透過 Testbench 以讀檔方式驗證全部 9 筆測試資料，輸出結果 100% 正確。

在設計規範方面，所有模組均嚴格遵守禁止使用 for/while、function/task、always@(*)、#delay 等規定。ALU 模組從最底層的 1-bit Full Adder 逐層建構，Shifter 以 160 個獨立 Mux 實現，Multiplier 以 First Version Sequential 架構完成 32-bit 無號數乘法，均符合課程要求的設計方式。

本次專題不僅達成了功能目標，更讓我在實作過程中建立了對硬體描述語言的基礎知識，以及組合邏輯與循序邏輯的設計方法差異。這些能力將為後續的期末專題（Final Project）打下重要基礎。

六、未來展望

本次期中專題作為 Final Project 的前置準備，在完成基礎 ALU 功能後，未來有以下幾個方向可以進一步延伸：

6.1 整合至完整 MIPS 處理器

本期中專題所設計的 ALU 系統將作為 Final Project 的核心運算單元，整合至完整的 MIPS 單週期或多週期處理器架構中，搭配指令記憶體、資料記憶體、Register File 與 Control Unit，實現完整的 Fetch-Decode-Execute 指令執行流程。

6.2 FPGA 實體驗證

本次專題僅在 ModelSim 模擬環境中驗證功能正確性。未來若有機會，可將設計下載至實際 FPGA 開發板進行實體驗證，觀察真實硬體電路的時序特性與資源使用量，進一步體驗從 RTL 設計到實際晶片的完整流程。

透過本次期中專題的學習過程，本人對計算機組織與數位電路設計有了更深入的理解，並建立了使用 Verilog HDL 進行硬體設計的實作能力，期待在 Final Project 中能夠進一步應用與延伸這些知識。

七、各組員分工

學號	姓名	負責項目
11327264	許明源	全部模組設計、驗證、報告撰寫（個人作業）